

DISTRIBUTED DATA PARADIGM SHIFT

Transitioning from MapReduce to PySpark

Evolving purely functional map, combine, and reduce patterns into
high-performance Spark execution



The Classic MapReduce Triad



map()

Transforms individual records into key-value intermediate tuples.



combine()

Performs local in-memory pre-aggregation before network shuffle.

reduce()

Aggregates partitioned values globally across key boundary groups.

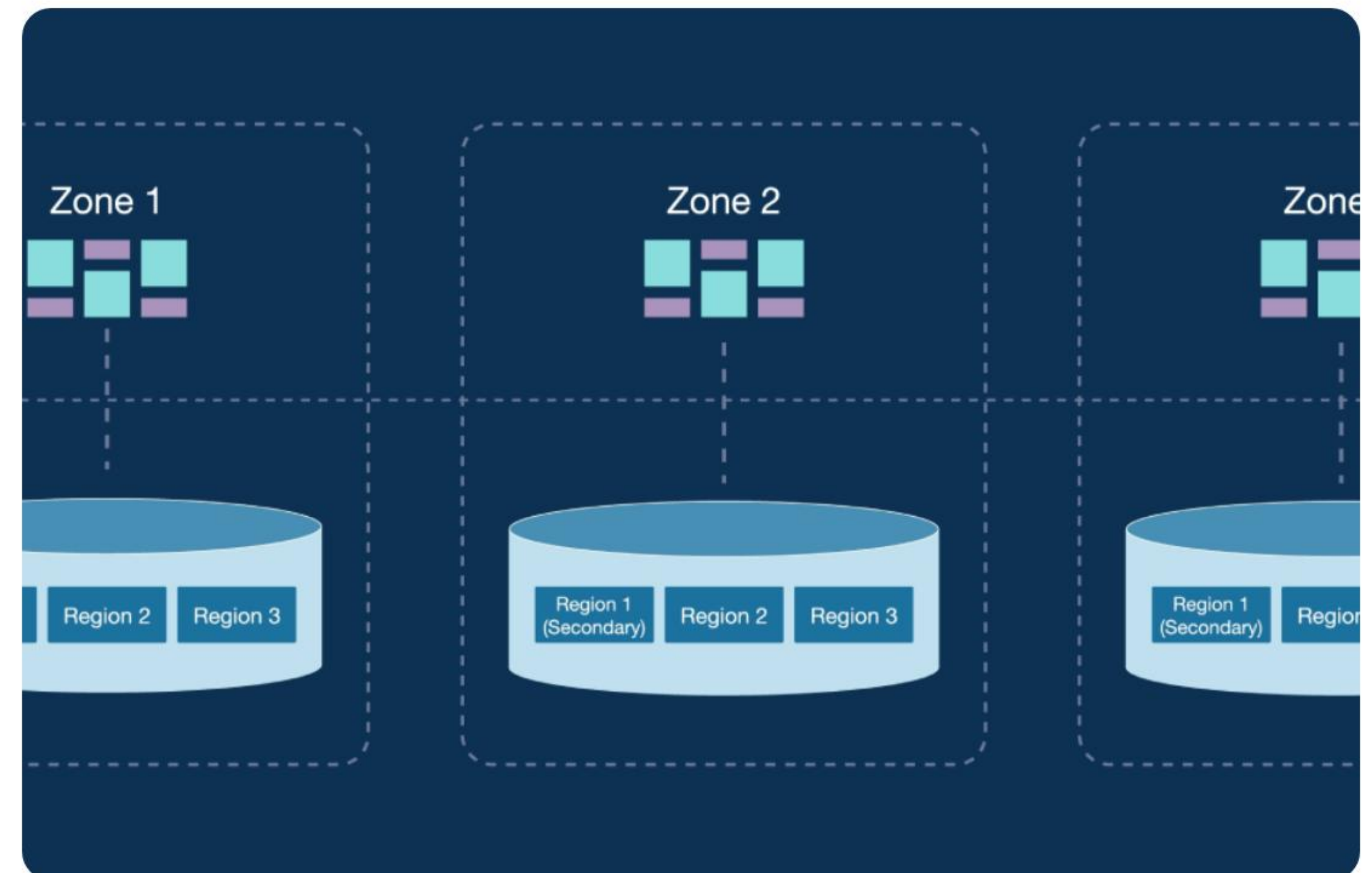
PARADIGM LIMITATIONS

Traditional MapReduce Bottlenecks

Disk Overhead: Mandatory intermediate disk writes between map and reduce stages.

High Latency: Strict barrier synchronization slows multi-step data pipelines.

Rigid Chaining: Complex workflows require manual multi-job stringing.



IN-MEMORY REVOLUTION

The Spark Core Advantage

100x

Faster in-memory computation compared to disk-bound execution

RAM Caching

In-memory state persistence across iterations.

Lazy Execution

Optimized DAG graphs compiled before execution.

Mapping Primitives to PySpark RDDs

Conceptual MapReduce

```
# Conceptual functional flow
words = map(split_words, lines)
pairs = map(make_pair, words)
combined = combine(local_sum, pairs)
result = reduce(global_sum, combined)
```

PySpark RDD Pipeline

```
# Idiomatic PySpark RDD
word_counts = (
    lines.flatMap(lambda line: line.split(" "))
           .map(lambda word: (word, 1))
           .reduceByKey(lambda a, b: a + b)
)
```

Replacing combine() with combineByKey()

```
# PySpark combineByKey for average calculation
rdd.combineByKey(
    createCombiner=lambda v: (v, 1),
    mergeValue=lambda acc, v: (acc[0] + v, acc[1] + 1),
    mergeCombiners=lambda a, b: (a[0] + b[0], a[1] + b[1])
)
```

Zero Spill Shuffle

Aggregates records locally before partition transfer.

Type Transformation

Converts raw inputs into distinct tuple accumulators.

Beyond RDDs: Modern Spark DataFrames

Low-Level RDDs

Explicit functional lambda transformations.

DataFrame API

Schema-aware declarative query operations.

Catalyst Engine

Automatic physical query plan optimization.

PySpark Execution Stage Flow

STAGE 1

Input Read

Loads data into initial RDD partitions.

STAGE 2

Lazy Transforms

Builds transformation lineage graph.

STAGE 3

Shuffle Exchange

Re-partitions data across network nodes.

STAGE 4

Action Trigger

Executes DAG and writes output result.

Architectural Paradigm Comparison

Feature	Classic MapReduce	PySpark RDD	PySpark DataFrame
Data Storage	Disk Storage	In-Memory RAM	In-Memory Off-Heap
Aggregation	combine() / reduce()	reduceByKey()	groupBy().agg()
Optimization	Manual Control	DAG Lineage	Catalyst Optimizer

SUMMARY & NEXT STEPS

Key Migration Takeaways

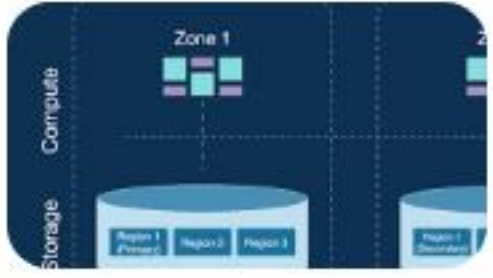
1. Combine Early: Use `reduceByKey()` or `combineByKey()` to reduce shuffle overhead.

2. Eliminate Disk I/O: Leverage Spark's in-memory caching for iterative computations.

3. Adopt DataFrames: Transition from low-level RDD operations to catalyst-optimized DataFrames.

Ready to transform your distributed pipelines?

Image Sources



<https://static.pingcap.com/files/2023/08/17235704/Distributed-SQL-Databases.png>

Source: www.pingcap.com